

ui.run_javascript 全面详解

`ui.run_javascript` 是 NiceGUI 框架中用于在浏览器端执行任意 JavaScript 代码的核心函数，支持同步 / 异步执行、元素操作、结果返回等多种场景，是连接 Python 后端与浏览器前端的关键桥梁。

一、核心功能与设计初衷

该函数的核心作用是在**客户端浏览器中执行自定义 JavaScript 代码**，解决 Python 无法直接操作浏览器端 DOM、API（如地理位置）等问题，实现前后端交互的灵活性：

- 可调用浏览器原生 API（如 `Date()`、地理定位）；
- 可操作页面 DOM 元素或 Vue 组件；
- 支持同步执行（无需等待结果）和异步执行（获取返回值）；
- 自动处理客户端连接状态（3.0.0+ 版本），确保代码在客户端就绪后执行。

二、函数参数与返回值

1. 关键参数

参数名	说明	默认值
<code>code</code>	必选参数，需要在浏览器中执行的 JavaScript 代码字符串（支持普通代码和异步代码）	-
<code>timeout</code>	可选参数，等待 JavaScript 执行结果的超时时间（单位：秒），超时会抛出异常	1.0

2. 返回值

返回 `AwaitableResponse` 对象，支持两种使用方式：

- 不等待结果**（同步执行）：直接调用函数，JavaScript 代码后台执行，Python 不阻塞；
- 等待结果**（异步执行）：通过 `await` 关键字获取 JavaScript 代码的执行结果（如数据、DOM 操作结果等）。

三、核心特性与注意事项

1. 客户端连接依赖

- 3.0.0+ 版本会自动调用 `await client.connected()`，确保代码在客户端与后端建立连接后执行；
- 该连接等待逻辑**不受** `timeout` **参数控制**，仅用于保证执行时机，超时仅针对 JavaScript 代码本身的执行过程。

2. 元素访问支持（2.9.0+ 版本）

框架提供两个内置 JavaScript 辅助函数，用于通过 ID 操作页面元素：

- `getElement(id)`：获取 Vue 组件实例（适用于 NiceGUI 内置组件，如 `ui.button`、`ui.label`）；
- `getHtmlElement(id)`：获取原生 HTML 元素（适用于直接操作 DOM 特性，如 `innerText`、`style`）。

- 注意：需先获取 NiceGUI 组件的 `id` 属性（如 `label.id`），再传入 JavaScript 代码中。

3. 异步 JavaScript 支持

浏览器端的异步操作（如地理定位、定时器、网络请求）可通过 `Promise` 封装，`ui.run_javascript` 能通过 `await` 捕获 `Promise` 的 `resolve` 结果或 `reject` 异常。

四、使用场景与示例代码

场景 1：无返回值执行（Fire and Forget）

适用于无需获取结果的简单操作（如弹窗、DOM 修改），Python 代码不阻塞。

```
from nicegui import ui

def alert():
    # 执行 JavaScript 弹窗，无需等待结果
    ui.run_javascript('alert("Hello from JavaScript!")')

ui.button('触发弹窗', on_click=alert)
ui.run()
```

场景 2：获取执行结果（Await Result）

适用于需要获取浏览器端数据的场景（如当前时间、用户输入），通过 `await` 接收返回值。

```
from nicegui import ui

async def get_browser_time():
    # 执行 JavaScript 的 Date() 方法，等待结果返回
    browser_time = await ui.run_javascript('Date()')
    ui.notify(f'浏览器当前时间: {browser_time}')

ui.button('获取浏览器时间', on_click=get_browser_time)
ui.run()
```

场景 3：操作页面元素

通过 `getHtmlElement` 操作 DOM 元素（如修改标签文本），需先获取组件 ID。

```
from nicegui import ui

def modify_label():
    # 给标签添加文本，label.id 是 NiceGUI 组件的内置属性
    ui.run_javascript(f'getHtmlElement({label.id}).innerText += " 你好!"')

label = ui.label('初始文本')
ui.button('修改标签', on_click=modify_label)
ui.run()
```

场景 4：执行异步 JavaScript（如地理定位）

封装浏览器异步 API（如地理定位）为 `Promise`，通过 `await` 获取结果，支持自定义超时。

```
from nicegui import ui

async def show_user_location():
    try:
        # 执行异步 JavaScript 代码，超时设置为 5 秒
        location_data = await ui.run_javascript('''
            return await new Promise((resolve, reject) => {
                // 检查浏览器是否支持地理定位
                if (!navigator.geolocation) {
                    reject(new Error('浏览器不支持地理定位'));
                    return;
                }
                // 调用地理定位 API
                navigator.geolocation.getCurrentPosition(
                    (position) => {
                        // 成功：返回经纬度
                        resolve({
                            latitude: position.coords.latitude,
                            longitude: position.coords.longitude
                        });
                    },
                    () => {
                        // 失败：抛出异常
                        reject(new Error('无法获取位置信息'));
                    }
                );
            });
        ''', timeout=5.0)
        # 显示结果
        ui.notify(f'你的位置: {location_data["latitude"]}, {location_data["longitude"]}')
    except Exception as e:
        ui.notify(f'错误: {str(e)}', color='red')

ui.button('获取位置', on_click=show_user_location)
ui.run()
```

五、版本兼容性说明

版本	新增特性
2.9.0	新增 <code>getElement()</code> 和 <code>getHtmlElement()</code> 辅助函数，支持通过 ID 访问元素
3.0.0	自动调用 <code>await client.connected()</code> ，确保客户端连接后执行代码

六、常见问题与注意事项

- 跨域限制：**JavaScript 代码受浏览器跨域策略限制，无法直接调用非同源的后端 API（需通过 NiceGUI 后端代

- 理)；
2. **超时设置**：异步场景需根据操作复杂度调整 `timeout`（如地理定位、大文件处理需延长超时）；
 3. **错误处理**：JavaScript 中的异常（如 `reject`、语法错误）会被 Python 捕获为异常，建议通过 `try-except` 处理；
 4. **组件 ID 唯一性**：操作元素时需确保组件 `id` 唯一（NiceGUI 自动生成唯一 ID，无需手动设置）。

总结

`ui.run_javascript` 是 NiceGUI 中实现前后端灵活交互的核心工具，既支持简单的 DOM 操作和弹窗，也能整合浏览器原生异步 API，通过同步 / 异步两种执行模式满足不同场景需求。使用时需注意版本兼容性、客户端连接状态和超时控制，结合 `await` 和辅助函数可高效实现复杂前端交互逻辑。